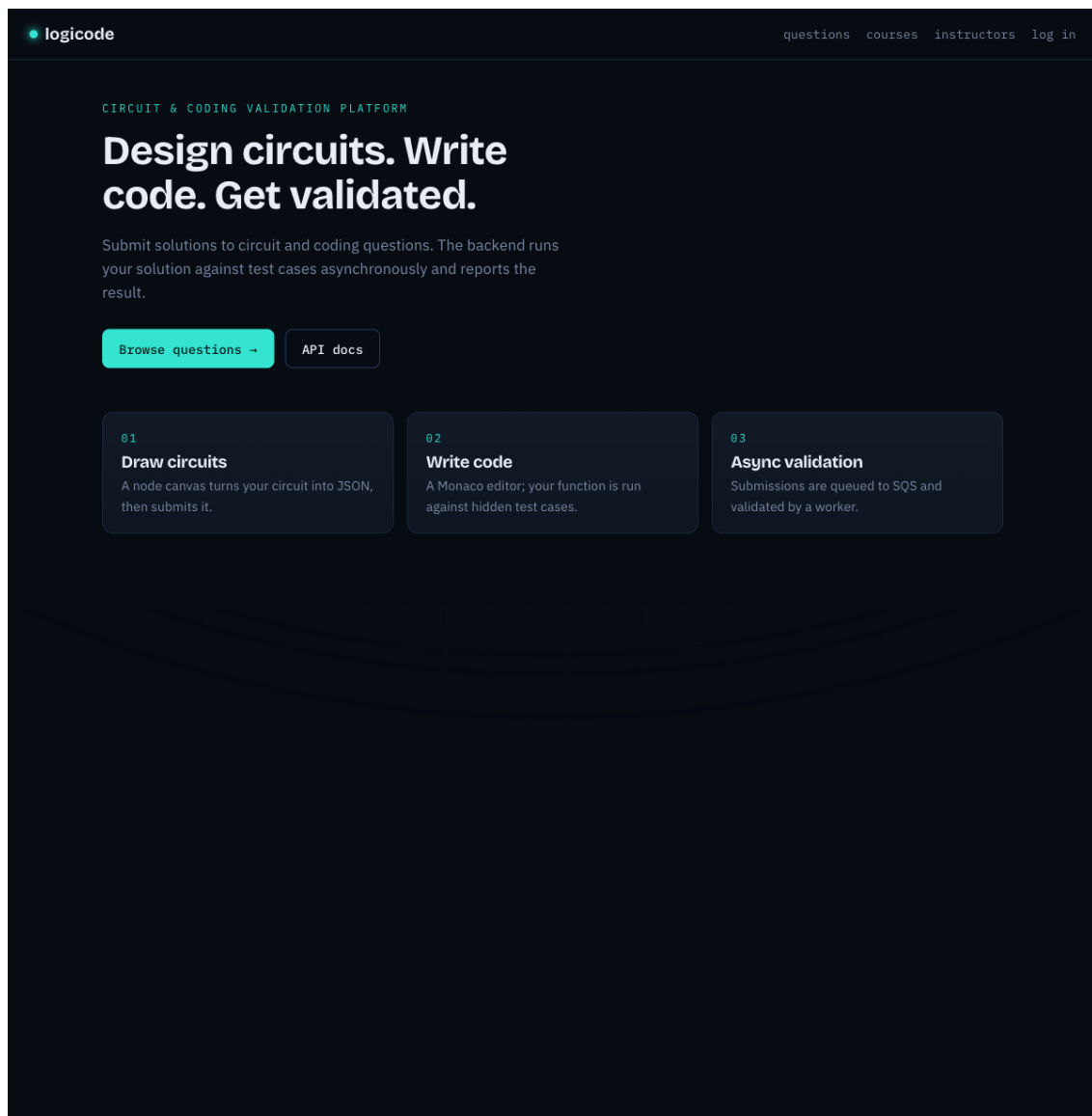

Logiccode

A platform for circuit and coding challenges with async AWS validation



June 2026

Contents

1	Problem & Context	2
2	Solution Overview	2
3	System Architecture	2
3.1	Infrastructure Diagram	3
3.2	Request Flow	3
3.3	Prototype Cost Decisions	3
4	Feature Walkthrough	4
4.1	Question Catalogue	4
4.2	Circuit Challenge	5
4.3	Coding Challenge	6
4.4	Courses	7
4.5	Instructors	9
4.6	Submission History	10
4.7	Admin Dashboard	11
4.8	API Documentation	12
5	Tech Stack	12
5.1	Frontend	12
5.2	Backend / Data	13
5.3	Infrastructure	13
6	CI/CD Pipeline	13
7	Delivery & Handoff	13

1 Problem & Context

Engineering platforms that teach digital logic and programming share a common friction point: students need immediate, reliable feedback on their solutions. Traditional approaches either rely on instructor grading (slow, not scalable) or run user-submitted code directly on the server (a security and resource risk).

Logicode addresses this by separating the **web experience** from **code execution**. Users compose solutions in the browser — drawing logic-gate circuits on a canvas or writing code in an embedded editor — and the platform validates asynchronously through a queue-driven Lambda pipeline, keeping the web tier lean and safe.

| The platform doubles as a full-stack AWS prototype: every infrastructure component is provisioned with Terraform, CI/CD runs on GitHub Actions with OIDC, and the entire system can be torn down and rebuilt from code.

2 Solution Overview

Logicode offers two question types:

- **Circuit challenges** — users wire logic gates (AND, OR, NOT, XOR, NAND, NOR) on an interactive React Flow canvas. The circuit is serialized to JSON and structurally compared against the expected solution.
- **Coding challenges** — users write JavaScript, Python, or Ruby solutions in a Monaco editor. Code is executed in isolated Lambda runtimes (one per language) and tested against hidden test cases.

Alongside the challenge engine, the platform includes a **course catalogue** (video-backed, Udemy-style accordion sections with enrollment gating), an **instructor directory** with booking, and a full **admin dashboard** for managing questions, courses, and instructors.

Authentication is handled by **Auth.js v5** (credentials + JWT + bcrypt), with user self-registration and a protected admin role.

3 System Architecture

3.1 Infrastructure Diagram

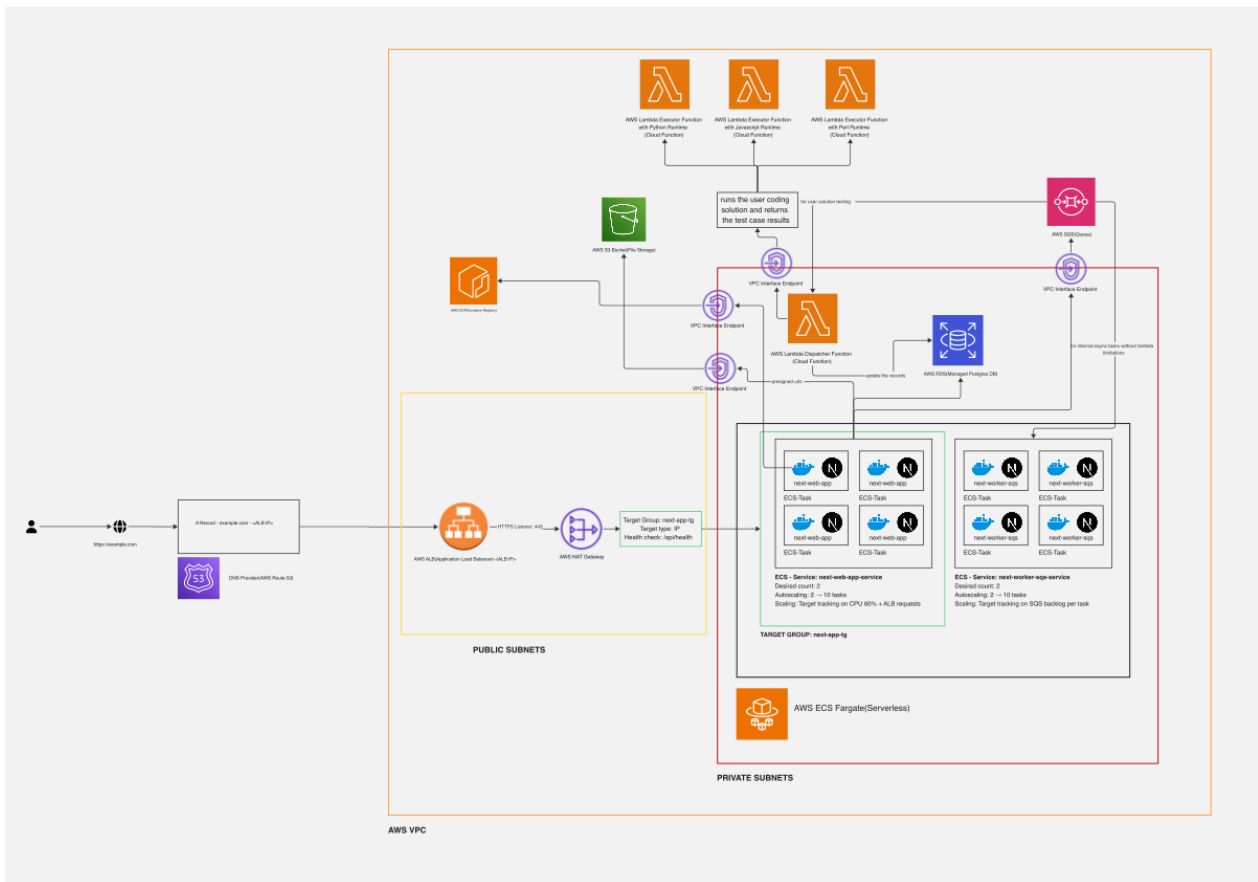


Figure 1 Target architecture: ALB → ECS Fargate → Lambda validators with SQS

3.2 Request Flow

1. User submits a solution via the browser.
2. The Next.js API stores the submission as PENDING in **RDS PostgreSQL** and enqueues a message to **SQS**.
3. The **dispatcher Lambda** (in-VPC, private subnet) consumes the message, fetches the submission and test cases from RDS, and *synchronously invokes* the appropriate per-language executor Lambda.
4. The executor Lambda runs the user's code in an isolated sandbox, evaluates each test case, and returns the result.
5. The dispatcher writes PASSED/FAILED + per-case breakdown back to RDS. The browser polls for completion.

3.3 Prototype Cost Decisions

The full target diagram is production-grade (multi-AZ, NAT, multi-env, \$100+/month). The prototype stays below \$40/month by using:

- **No NAT Gateway** — ECS tasks run in public subnets with public IPs; RDS remains private.
- **Lambda VPC Interface Endpoint** (~\$7/month) — enables dispatcher (private subnet) to reach executor Lambdas without a NAT.
- **Single environment** (dev); single ECS task, smallest RDS instance (db.t4g.micro).

4 Feature Walkthrough

4.1 Question Catalogue

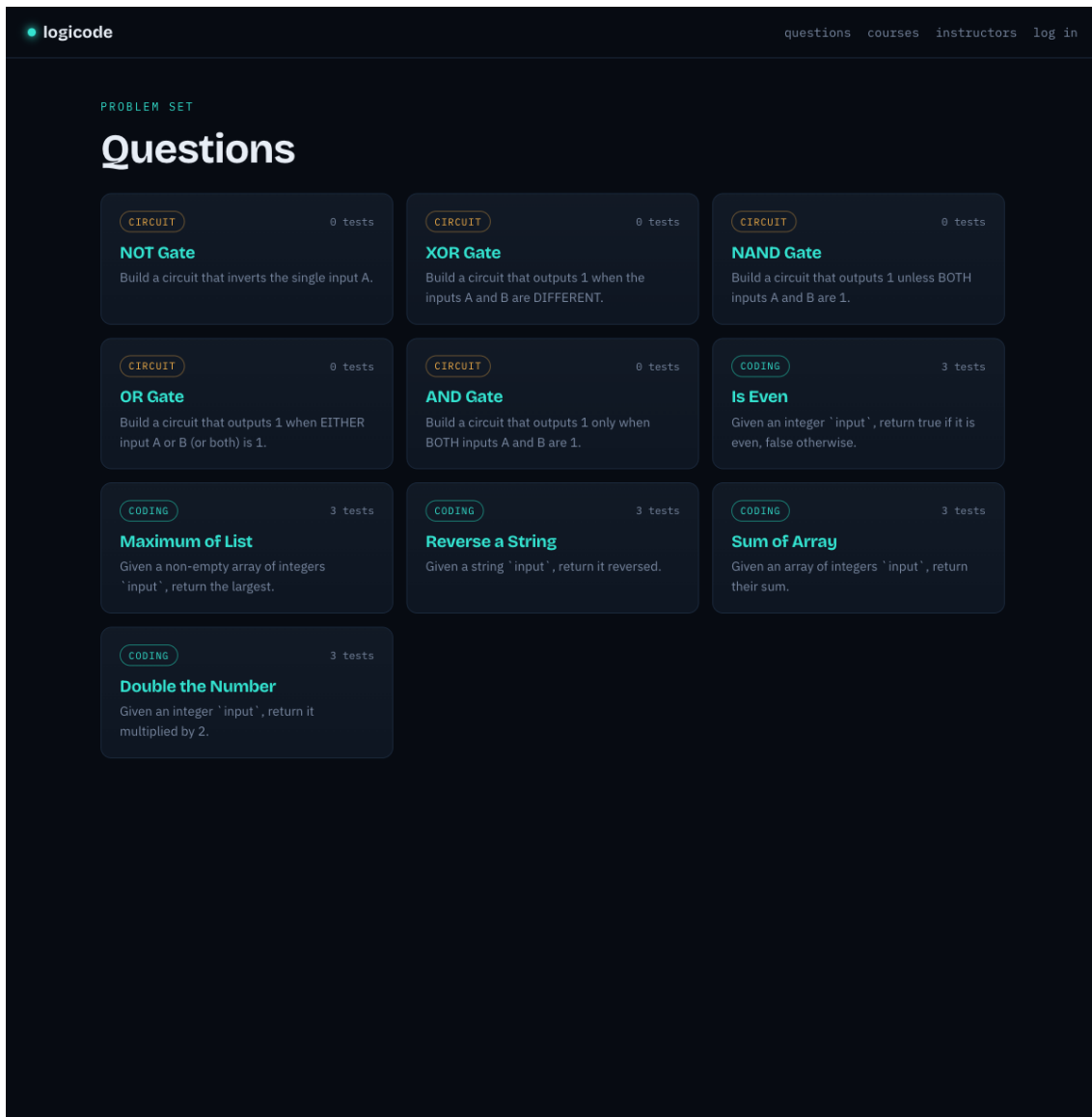


Figure 2 Question catalogue — CIRCUIT and CODING cards with test-case counts

Questions are tagged by type (CIRCUIT or CODING) and show the number of test cases. All questions are accessible without login; submitting a solution requires authentication.

4.2 Circuit Challenge

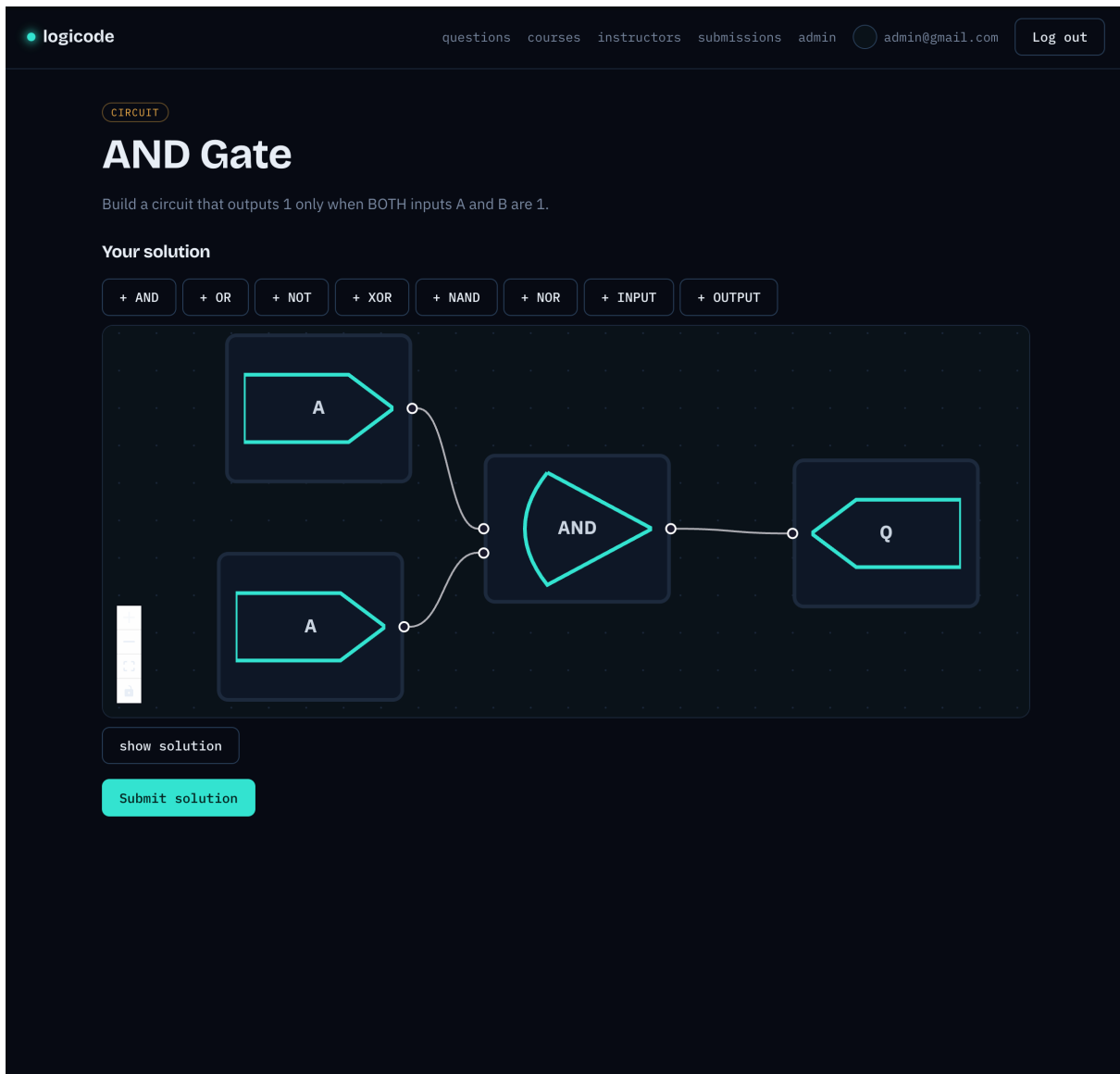


Figure 3 AND Gate challenge — React Flow canvas with gate palette and submit panel

The circuit editor is built on **React Flow** (@xyflow/react). Users drag gates from the component palette onto the canvas, connect them with edges, and hit *Submit*. The frontend serializes the graph to JSON; the Lambda validator performs a structural comparison (gate-type multiset, edge count, output-driven traversal) against the stored solution.

4.3 Coding Challenge

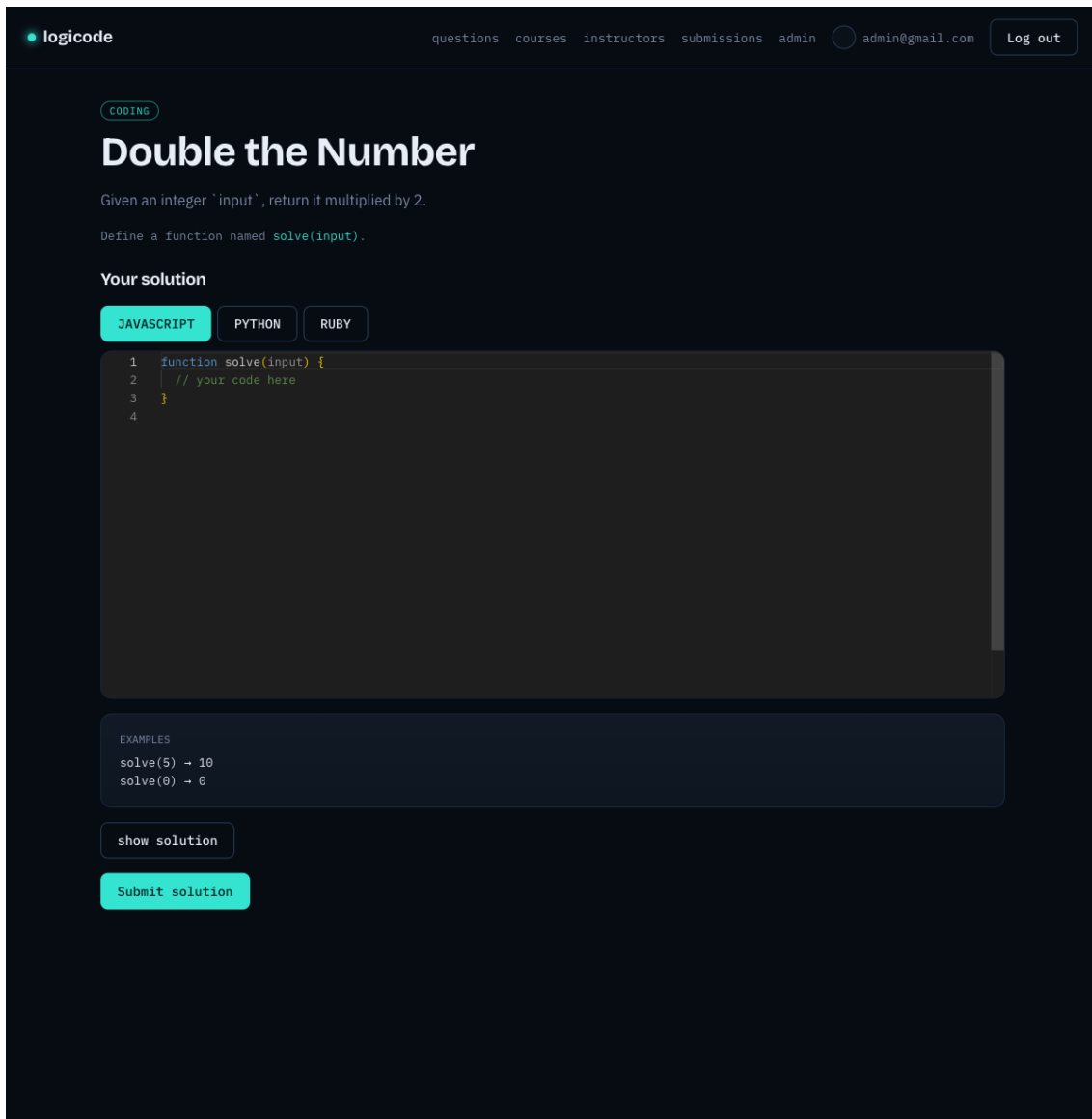


Figure 4 Coding challenge — Monaco editor with language selector (JS / Python / Ruby)

Each coding question ships per-language boilerplate (CodeTemplate model). Users pick their language, edit the solution in the embedded **Monaco editor**, and submit. The dispatcher routes the submission to the matching executor Lambda (Node.js node:vm, Python subprocess, Ruby subprocess), which runs the function against each test case and returns a per-case pass/fail.

4.4 Courses

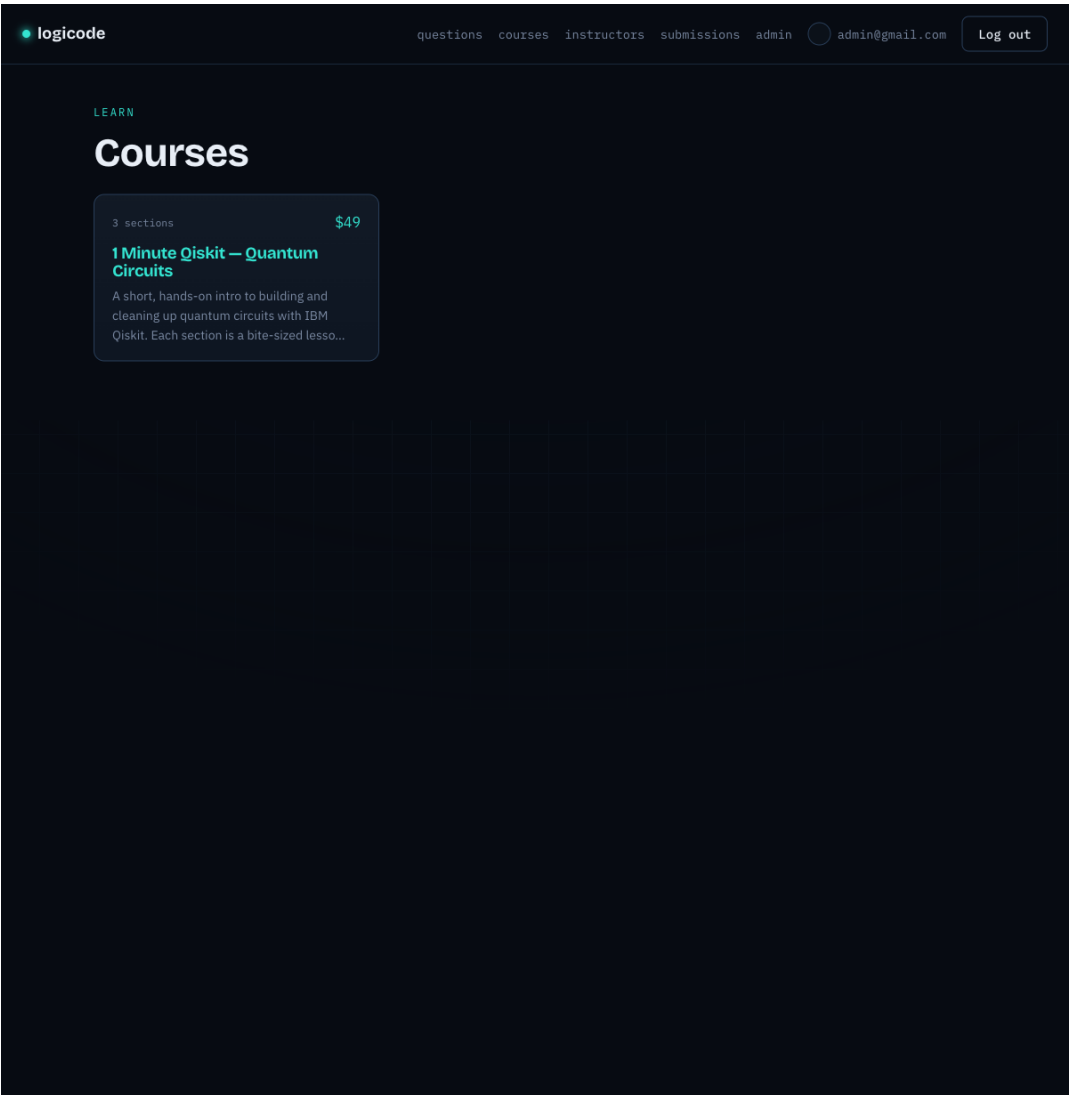


Figure 5 Course catalogue with section count and price badge


[questions](#)
[courses](#)
[instructors](#)
[submissions](#)
[admin](#)
admin@gmail.com
[Log out](#)

COURSE

1 Minute Qiskit — Quantum Circuits

\$49

A short, hands-on intro to building and cleaning up quantum circuits with IBM Qiskit. Each section is a bite-sized lesson.

FREE PREVIEW



0:01 / 3:09

Get full access

Enroll to unlock all 3 sections. Only the free preview is available until then.

Enroll — \$49

Course content • 3 sections

01 Removing gates from a circuit

Enroll to watch this section.

Figure 6 Course detail — Udemy-style accordion sections, enrollment gating, video preview

Courses are structured as a set of `CourseSection` records. Videos are stored in **S3** and served via presigned GET URLs (no proxying through the app). Preview sections are accessible before enrollment; full content unlocks after `POST /api/courses/:id/enroll`.

4.5 Instructors

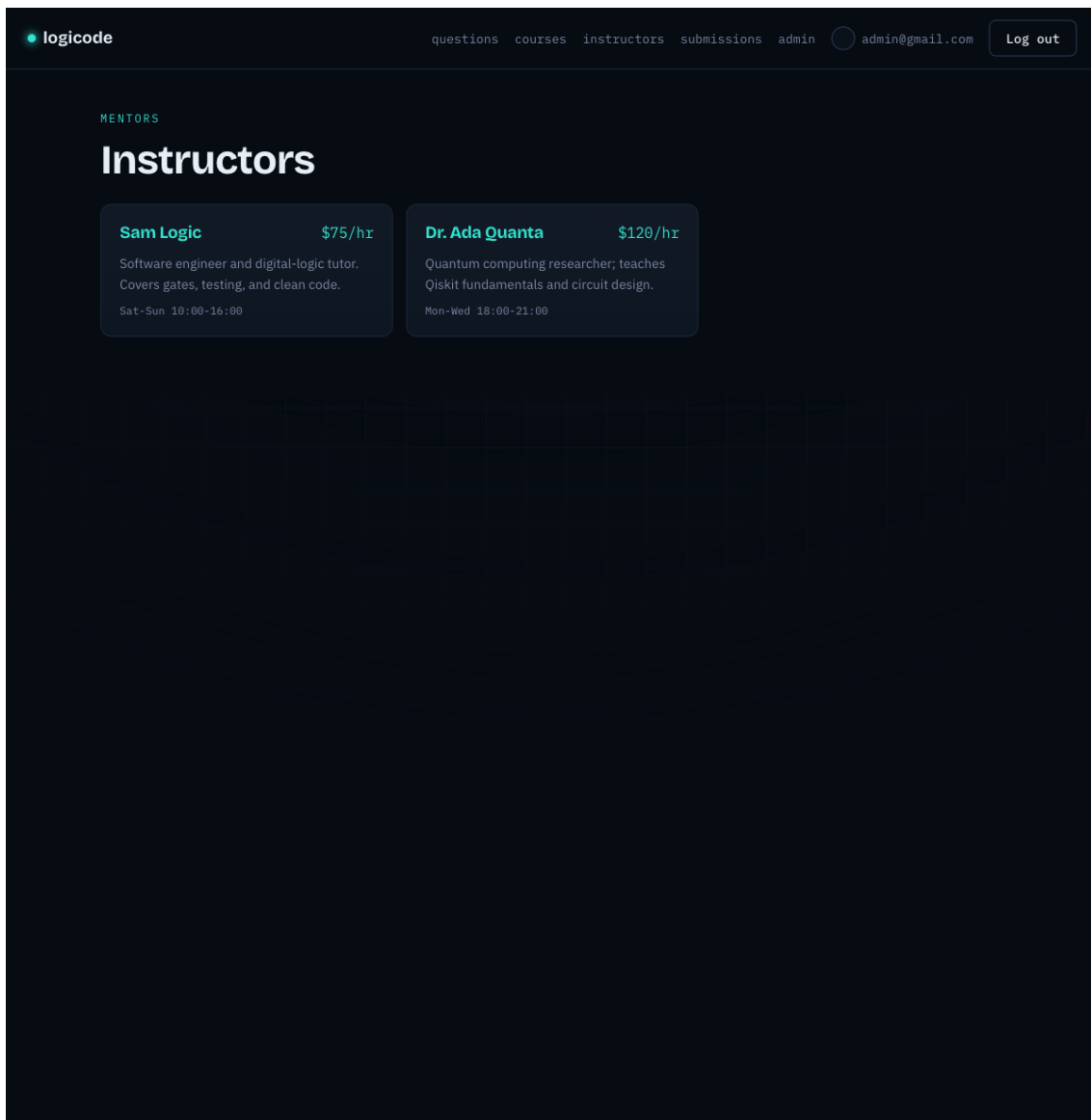


Figure 7 Instructor directory with avatar, bio, and booking action

Each instructor card links to a booking flow (`POST /api/instructors/:id/bookings`). Instructor avatars are uploaded by admins, stored in S3, and displayed via presigned GET URLs. Users can also upload a personal avatar from their `/account` page.

4.6 Submission History

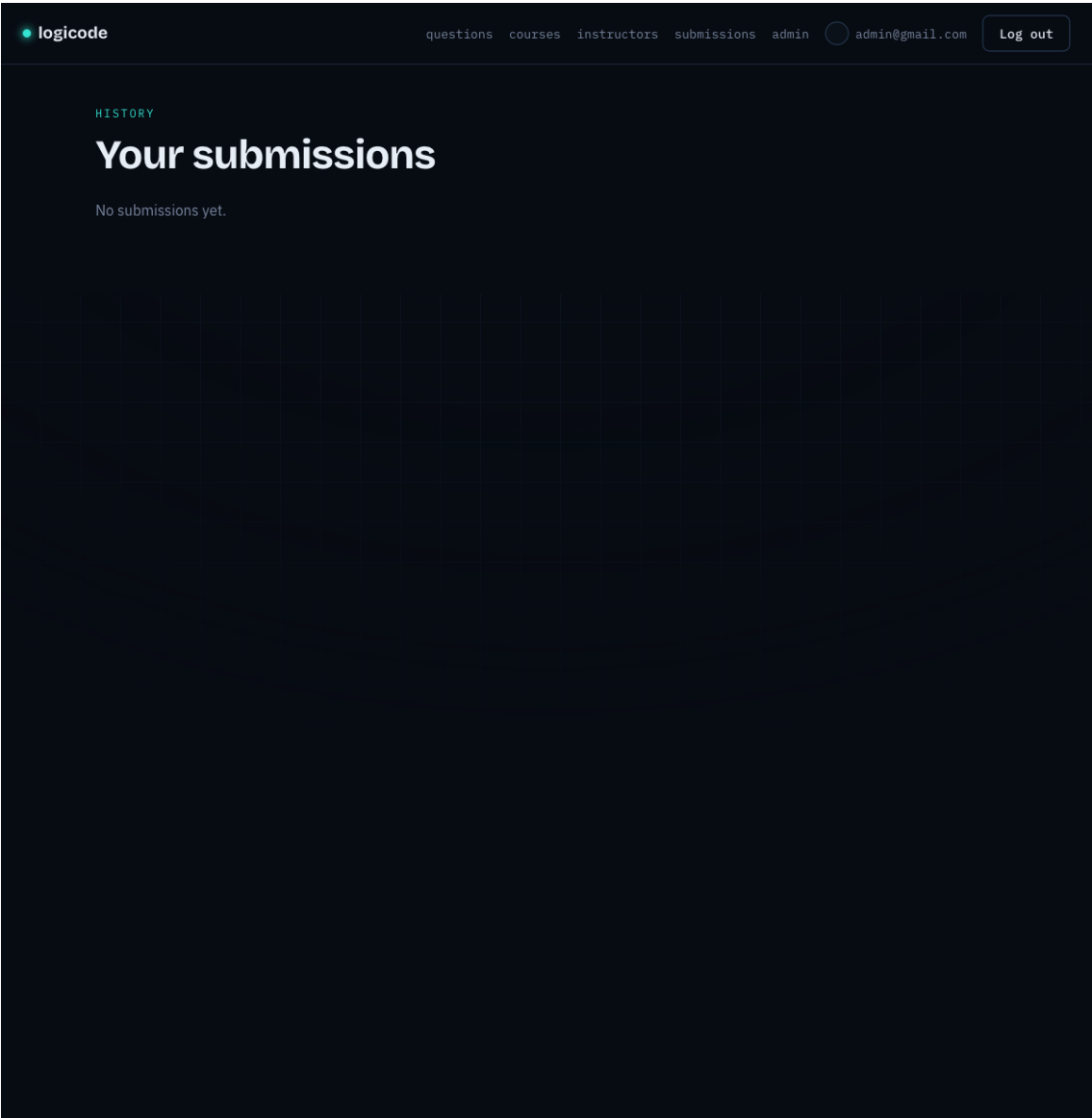


Figure 8 Submission list — status badges, language, question link, detail drill-down

Every submission is stored with its PENDING → PASSED/FAILED lifecycle. The list links to a `/submissions/:id` detail page showing the full payload and per-test-case breakdown. Admins see all submissions; regular users see only their own.

4.7 Admin Dashboard

The screenshot shows the Logicode Admin Dashboard. At the top, there's a navigation bar with links for 'questions', 'courses', 'instructors', 'submissions', 'admin', and a 'Log out' button. The 'admin' tab is active. Below the navigation bar, there are three tabs: 'Questions', 'Instructors', and 'Courses'. The 'Questions' tab is selected, showing a 'New question' form. The form includes fields for 'TITLE', 'DESCRIPTION', 'TYPE' (set to 'CODING'), and 'ENTRY FUNCTION NAME' (set to 'solve'). Below these fields, there's a 'Language' selector with options for 'JAVASCRIPT', 'PYTHON', and 'RUBY'. The 'JAVASCRIPT' option is selected. The form also includes a 'JAVASCRIPT' section with a 'SOLUTION' field containing a sample solution in JavaScript. Below the solution, there's a 'TEST CASES' section with a table for 'INPUT (JSON)' and 'EXPECTED (JSON)'. The table has one row with '5' as input and '10' as expected output. There's a '+ add test case' button and a 'Create question' button. Below the form, there's a section for 'Existing questions' with a list of questions, each with an 'edit' and 'delete' button. The questions listed are: 'NOT Gate', 'XOR Gate', 'NAND Gate', 'OR Gate', 'AND Gate', 'Is Even - solve()', 'Maximum of List - solve()', 'Reverse a String - solve()', 'Sum of Array - solve()', and 'Double the Number - solve()'.

Figure 9 Admin dashboard — Questions, Instructors, Courses tabs with inline CRUD

The admin panel provides a three-tab interface for managing all content:

- **Questions** — create/edit/delete questions with per-language Monaco boilerplate, sample solutions, and inline test case management.
- **Instructors** — add instructors with avatar upload.
- **Courses** — create courses with sections and video upload.

4.8 API Documentation



Figure 10 Swagger UI at /docs — auto-generated from route annotations

The REST API is documented with **next-swagger-doc** and exposed at /docs via Swagger UI. All endpoints covering questions, submissions, courses, instructors, uploads, and health are included.

5 Tech Stack

5.1 Frontend

- **Next.js 16** (App Router, TypeScript) — server components, API routes, incremental rendering.
- **Monaco Editor** — VS-Code-grade code editor embedded in the browser.
- **React Flow (@xyflow/react)** — node-graph canvas for circuit building.
- **Auth.js v5** — credential-based authentication with JWT sessions.

5.2 Backend / Data

- **Prisma 7** with `@prisma/adapters-pg` (Rust-free driver adapter) on **PostgreSQL**.
- **Zod** — schema validation at API boundaries.
- **AWS Lambda** — three executor Lambdas (Node, Python, Ruby) plus the in-VPC dispatcher Lambda (Node, pg driver, no Prisma).
- **Amazon SQS** — decouples submission ingestion from validation.
- **Amazon S3** — file storage via presigned URLs (no server-side proxying).

5.3 Infrastructure

- **Terraform** — all AWS resources as IaC (VPC, ECR, ECS Fargate, ALB, RDS, S3, SQS, Lambda, ACM, SSM, CloudWatch, IAM/OIDC).
- **AWS ECS Fargate** — containerized Next.js app, autoscale 1–3 tasks on CPU 60%.
- **Cloudflare DNS** — DNS-only record pointing to the ALB; ACM certificate for TLS.
- **GitHub Actions** — CI (lint / typecheck / test / build) + deploy with OIDC (no static AWS keys).

6 CI/CD Pipeline

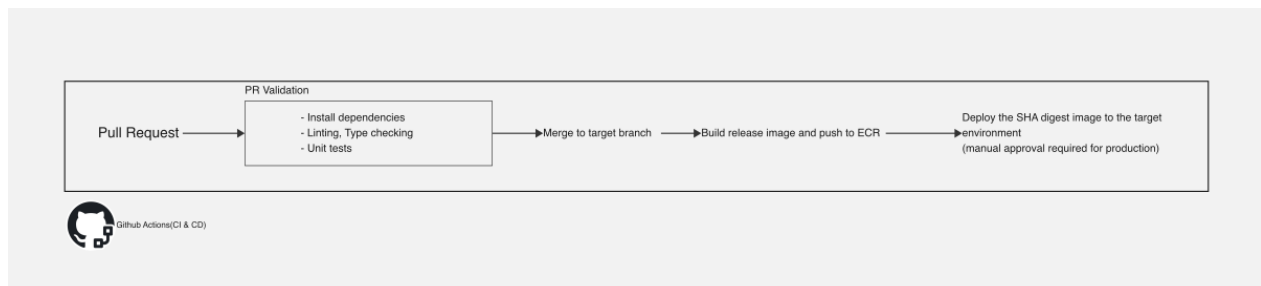


Figure 11 CI/CD pipeline: GitHub Actions → ECR → ECS Fargate via OIDC

Two GitHub Actions workflows run on this repository:

- **ci.yml** — triggered on PRs and pushes to dev/staging/main. Runs: `npm install`, `prisma generate`, `lint`, `typecheck`, `vitest`, `build`.
- **deploy.yml** — triggered on push to main. Builds the Docker image, pushes to **ECR** tagged with the commit SHA, and applies Terraform to update the ECS task definition to the new image.

Authentication uses **GitHub OIDC**: no long-lived AWS credentials are stored; the `cicd` Terraform module creates and scopes the IAM role that the workflow assumes.

7 Delivery & Handoff

The full source — including Terraform modules, Lambda code, GitHub Actions workflows, and the Next.js application — is organized for immediate handoff or further development.

All infrastructure is defined as code and can be deployed to a fresh AWS account in a single `scripts/deploy.sh` call, or destroyed completely with `scripts/destroy.sh`.

Built with Next.js · Prisma · AWS Lambda · ECS Fargate · Terraform · GitHub Actions